Model Evaluation – Understanding Sensitivity, Specificity, and Predictive Values

1. Intro

Imagine you've built a machine learning model that predicts whether a person has a disease — let's say diabetes. Your model gives results like *Positive* (they have diabetes) or *Negative* (they don't).

But here's the catch — accuracy alone can be misleading. A model can show **95% accuracy** and still fail miserably if the data is imbalanced.

So, how do we really know if our model is good?

That's where **Sensitivity, Specificity, False Positive and False Negative rates, PPV, and NPV** come in.

2. Concept Explanation

Step 1: The Confusion Matrix

Here's the foundation — the **confusion matrix**. It looks like this:

	Predicted Positive	**Predicted Negative**
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

Think of it like a report card of your model — showing what it got right and wrong.

Let's use an easy example.

Out of 100 people tested for diabetes:

10 actually have diabetes.
The model correctly finds 9 of them → **True Positives (TP = 9)**.
It misses 1 person → **False Negative (FN = 1)**.
Among 90 healthy people, it wrongly flags 5 as diabetic → **False Positive (FP = 5)**.
It correctly marks 85 as healthy → **True Negative (TN = 85).**

That's all we need to calculate the rest.

Step 2: Sensitivity / Recall

Sensitivity tells us — *out of all real positive cases, how many did we catch?*

$$\text{Sensitivity} = \frac{TP}{TP + FN}$$

In our example: 9 / (9 + 1) = 0.9 → **90% sensitivity.**

👉 High sensitivity means we rarely miss real positives.

	Predicted Positive	**Predicted Negative**
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

In healthcare, that's crucial — you don't want to miss someone who's actually sick.

Think of a smoke detector. A highly sensitive one will catch even a tiny bit of smoke — it might beep often, but at least it won't miss a fire.

Step 3: Specificity

Specificity asks — *out of all real negatives, how many did we correctly say are negative?*

$$\text{Specificity} = \frac{TN}{TN + FP}$$

In our example: 85 / (85 + 5) = 0.944 → **94.4% specificity.**
That means the model correctly identifies 94% of healthy people as healthy.

Analogy:
Specificity is like a security guard — it decides who *should not* enter.
A high-specificity guard won't wrongly stop innocent people.

Step 4: False Positive & False Negative Rates

Let's flip those formulas:

- **False Positive Rate (FPR)** = FP / (FP + TN)

It's the percentage of healthy people wrongly told they have diabetes.

- **False Negative Rate (FNR)** = FN / (FN + TP)
  It's the percentage of sick people the model failed to detect.

Both should ideally be **low** — but there's always a trade-off.
If you make your model more sensitive (catch more positives), you often increase false positives too.

	Predicted Positive	**Predicted Negative**
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

Step 5: PPV and NPV

Now, let's check if we can trust the model's predictions.

- **PPV (Positive Predictive Value)** or **Precision**

$$\text{PPV} = \frac{TP}{TP + FP}$$

Out of all predicted positives, how many were actually correct?
→ 9 / (9 + 5) = 0.64 or 64%.

- **NPV (Negative Predictive Value)**

$$\text{NPV} = \frac{TN}{TN + FN}$$

Out of all predicted negatives, how many were actually correct?
→ 85 / (85 + 1) = 0.988 or 98.8%.

So, our model is better at saying who's healthy than who's sick — a common issue in imbalanced datasets.

3. Mini Real-World Use Case

I. Job Application Screening AI

Scenario:
Imagine a company using an ML model to filter job applicants for an interview.

- **Positive = Suitable candidate**
- **Negative = Not suitable**

Now,

- A **True Positive** is a genuinely skilled candidate shortlisted.
- A **False Positive** is an unqualified candidate selected — wasting the interviewers' time.
- A **False Negative** is a great candidate rejected — a real loss.

	Predicted Positive	**Predicted Negative**
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

Insight:
A hiring model should have **high sensitivity** if the goal is to *not miss any good candidates*, but **high specificity** if you want to *avoid wasting interview slots* on weak applicants.

II. Autonomous Car – Pedestrian Detection

Scenario:
An ML model detects whether there's a pedestrian on the road.

- **Positive = Pedestrian detected**
- **Negative = No pedestrian**

Now:

- **False Negative (missed pedestrian)** can cause an accident.
- **False Positive (detects ghost pedestrian)** might unnecessarily brake and slow traffic.

Insight:
Here, **high sensitivity** is *critical for safety* — we can tolerate some false alarms, but we can't afford to miss a real person.

III. Face Unlock on Smartphones

Scenario:
Your phone uses facial recognition to unlock.

- **Positive = Authorized face detected**
- **Negative = Unauthorized face**
- **False Positive:** Someone else's face unlocks your phone — security breach.
- **False Negative:** Your phone refuses to unlock for you — annoying but safe.

Insight:
Face unlock systems favor **high specificity** (don't unlock for strangers), even if it means occasionally rejecting your face in poor light.

4. Summary

Let's recap:

- Sensitivity = how well the model catches positives.
- Specificity = how well it avoids false alarms.

- FPR/FNR = the types of mistakes it makes.
- PPV/NPV = how trustworthy its predictions are.